**ГБОУ школа №444**

# Проект ‹‹SlopShield›› - Blue Team

**Проект выполнил - Куликов Артём Дмитриевич**

**Оглавление**


- **Аннотация**
- **Цели и задачи**
    1. Анализ и моделирование угроз
    2. Проектирование архитектуры решения
    3. Разработка алгоритмов обнаружения (Anti-Slop, Anti-Typo)
    4. Разработка механизма генерации защищенного SBOM
    5. Выбор и обоснование технологического стека
    6. Создание полной проектной документации
- **Описание угроз безопасности**
- **Архитектура решения**
    1. Парсинг файлов зависимостей
    2. Проверка по локальному белому списку
    3. Детекция фантомов (Anti-Slop)
    4. Детекция typosquatting (Anti-Typo)
    5. Генерация SBOM
- **Технологический стек**
- **Результаты и форматы данных**
    - Схема данных Component
    - Схема данных SecureSBOM
- **Приложения**
    - Приложение 1: Пример использования утилиты в CI/CD-пайплайне
    - Приложение 2: Полный протофайл slopushield.proto
    - Приложение 3: Таблица сравнения подходов к детекции угроз

# Аннотация

Данный исследовательский проект посвящен разработке и анализу специализированного инструмента защиты цепочек поставок программного обеспечения — утилиты командной строки «SlopShield». Проект выполнен в рамках Всероссийской олимпиады школьников (ВсОШ) 2026 по направлению «Blue Team». Основная цель утилиты — автоматизация процесса аудита зависимостей в проектах на языках Python и Node.js для выявления двух перспективных угроз: slopsquatting и typosquatting.

Современные практики разработки программного обеспечения все чаще используют искусственный интеллект для помощи в написании кода, что привело к возникновению новой формы атак — slopsquatting . В этом [12] [13]
случае ИИ-ассистенты могут рекомендовать или генерировать код с вызовами несуществующих пакетов, которые злоумышленники заранее регистрируют и заполняют вредоносным кодом . Традиционные сканеры уязвимостей (SCA), [15]
такие как Snyk или OWASP Dependency-Check, не способны обнаружить такую угрозу, поскольку они ориентированы на поиск известных уязвимостей (CVE) в уже существующих и загруженных пакетах . «SlopShield» решает эту проблему, [6]
применяя проактивный подход, который проверяет сам факт существования пакета и его происхождение.

Typosquatting является более старой, но все еще актуальной угрозой, заключающейся в регистрации пакетов с опечатками в названиях популярных зависимостей . Для борьбы с этим механизмом «SlopShield» реализует [32] [40]
нечеткий поиск на основе расстояния Дамерау-Левенштейна, позволяющий эффективно находить подобные пакеты даже при наличии нескольких ошибок в имени . [46]

Архитектура утилиты спроектирована таким образом, чтобы она могла быть интегрирована в качестве надежного «проверяющего барьера» в конвейеры CI/ CD . Она работает перед этапом сборки проекта, анализируя файлы [36]
зависимостей (requirements.txt, package.json). Логика анализа многоступенчата: сначала выполняется проверка по локальному белому списку доверенных пакетов, затем для каждого пакета осуществляется запрос к API репозитория PyPI/npm. Если пакет не найден (статус 404), это немедленно помечается как критическая угроза, свидетельствующая о возможном slopsquatting. Если же пакет существует, система проверяет его возраст; пакеты, созданные менее недели назад, помечаются как подозрительные, требующие

карантинирования. Наконец, для каждой найденной зависимости выполняется нечеткий поиск ее имени в базе популярных пакетов для обнаружения typosquatting.

Все результаты аудита упаковываются в Software Bill of Materials (SBOM) — состав программного обеспечения. Ключевой особенностью «SlopShield» является генерация зашифрованного SBOM в бинарном формате Google Protocol Buffers (Protobuf). Такой подход обеспечивает не только высокую скорость и компактность, но и повышает отказоустойчивость от подделки SBOM внутри взломанной среды . Для шифрования используется современный и [22]
безопасный режим AES-GCM, который гарантирует как конфиденциальность, так и целостность данных . Таким образом, «SlopShield» представляет собой [5] [10]
комплексное решение, которое не только обнаруживает новые угрозы, но и обеспечивает надежное хранение и защиту данных о составе программного обеспечения.

# Цели и задачи

Основной целью данного проекта является создание высокоэффективного и инновационного инструмента защиты цепочек поставок программного обеспечения, способного противостоять новым видам атак, связанных с зависимостями, которые не могут быть обнаружены традиционными средствами контроля. Проект разрабатывается в строгом соответствии с требованиями и форматом Всероссийской олимпиады школьников (ВсОШ) 2026 по направлению «Blue Team», что предполагает акцент на практической ценности, глубине технического анализа и адекватности решаемой проблемы.
Целью проекта является разработка утилиты командной строки «SlopShield», которая будет выполнять функции надежного проверяющего барьера в процессах непрерывной интеграции и доставки, предотвращая интеграцию вредоносных или потенциально опасных зависимостей в исходный код проекта до этапа сборки.

Для достижения этой цели были сформулированы следующие конкретные задачи:

1. **Анализ и моделирование угроз:** Провести всесторонний анализ текущего состояния угроз безопасности цепочек поставок, сфокусировавшись на двух ключевых проблемах: slopsquatting, возникающем из-за галлюцинаций ИИ-ассистентов, и typosquatting, связанном с опечатками в именах пакетов. Необходимо было точно определить механизмы этих атак, их последствия и то, почему существующие инструменты их не покрывают.

2. **Проектирование архитектуры решения:** Разработать архитектуру утилиты, которая позволяет интегрировать ее в качестве «Gatekeeper» в CI/CD конвейеры . Это потребовало создания многоступенчатого конвейера [3][6] анализа, способного обрабатывать зависимости из различных экосистем (Python, Node.js) и принимать окончательное решение (пропустить сборку или заблокировать ее) на основе комплексной оценки рисков.

3. **Разработка алгоритмов обнаружения:** Реализовать два основных механизма обнаружения угроз:

- **Anti-Slop (Обнаружение фантомов):** Создать логику, которая проверяет существование каждого пакета в официальных репозиториях (PyPI, npm). Алгоритм должен реагировать на ответ HTTP 404 Not Found как на критическую угрозу slopsquatting и дополнительно анализировать метаданные пакета (например, дату загрузки), чтобы выявлять свежие, подозрительные пакеты, созданные специально для атаки.
- **Anti-Typo (Обнаружение опечаток):** Интегрировать алгоритм нечеткого строкового сравнения на основе расстояния Дамерау-Левенштейна для идентификации пакетов, зарегистрированных с опечатками в названиях популярных зависимостей . Была выбрана библиотека C [4][6] оптимизированная Levenshtein для обеспечения высокой производительности.

1. **Разработка механизма генерации защищенного SBOM:** Спроектировать и реализовать механизм генерации SBOM (состава программного обеспечения), который бы не только предоставлял информацию о зависимостях, но и был защищен от подделки. Это достигается путем использования бинарного формата Protobuf для компактности и скорости, а также шифрования итогового файла с помощью криптографического алгоритма AES-GCM для обеспечения целостности данных (подтверждение подлинности и конфиденциальности) . [5][10]

2. **Выбор и обоснование технологического стека:** Определить и обосновать выбор всех технологий, используемых в проекте. Это включает язык программирования (Python 3.10+), систему сериализации данных (Google Protocol Buffers), библиотеку криптографии (cryptography) и библиотеки для текстовых операций (Levenshtein/RapidFuzz). Каждый выбор должен быть подкреплен техническими и практическими аргументами, демонстрирующими глубокое понимание предметной области.

3. **Создание полной проектной документации:** Оформить все результаты исследования, проектирования и разработки в соответствии с официальным шаблоном проектной записки ВсОШ 2026. Документ должен быть структурирован, содержать все необходимые разделы (титульный лист, оглавление, аннотацию, цели, описание угроз, архитектуру, технологический стек, результаты, приложения) и представлять собой завершенную и убедительную работу.

Выполнение этих задач позволило создать полноценный прототип утилиты «SlopShield», демонстрирующий оригинальный подход к защите цепочек поставок и решающий конкретную, актуальную проблему в области кибербезопасности.

# Описание угроз безопасности

В современной экосистеме разработки программного обеспечения цепочка поставок представляет собой сложную и многокомпонентную систему, где каждый элемент зависит от множества сторонних библиотек и фреймворков. Защита этого периметра стала одной из ключевых задач в области кибербезопасности, поскольку компрометация одного маленького звена может привести к масштабным инцидентам на уровне всей организации . Проект [18] «SlopShield» нацелен на противодействие двум конкретным, но различным по своей природе угрозам, которые активно используются злоумышленниками для подмены легитимных зависимостей вредоносными аналогами: slopsquatting и typosquatting.

Первой и главной угрозой, являющейся основной движущей силой проекта, является **slopsquatting**. Этот термин описывает новый метод кибератаки, возникший на стыке развития больших языковых моделей (LLM) и практик управления зависимостями в ПО . Современные ИИ-ассистенты, такие как [12] [27] GitHub Copilot или ChatGPT, способны генерировать код, включающий вызовы к внешним библиотекам, которые на самом деле не существуют в общедоступных репозиториях . Злоумышленники, осведомленные об этом явлении, заранее [14] регистрируют эти "галлюцинируемые" имена пакетов в репозиториях PyPI (для Python) или npm (для JavaScript) и загружают в них вредоносный код . Когда [13] разработчик, используя ИИ-помощника, добавляет такой пакет в свой проект, он inadvertently интегрирует вредоносное ПО в свою систему . Традиционные [15] сканеры уязвимостей (Software Composition Analysis, SCA), такие как Snyk, Sonatype Nexus Lifecycle или OWASP Dependency-Check, бесполезны против этой угрозы, поскольку они предназначены для поиска известных уязвимостей (CVE) в уже существующих и загруженных пакетах . Пакет, созданный специально [6] для атаки, будет технически "чистым", не имея ни одного известного CVE, но при этом представлять прямую угрозу. Эта угроза классифицируется в рамках стандартной

матрицы MITRE ATT&CK под техникой T1195.001 — Compromise Software Dependencies (Компрометация зависимостей программного обеспечения), что корректно отражает ее суть как часть атаки на цепочку поставок . [8] [11]

Второй угрозой является **typosquatting**, которая, хотя и является более старой проблемой, остается актуальной и представляет собой серьезный риск . [3] [2]

Typosquatting — это техника, при которой злоумышленники регистрируют пакеты с именами, похожими на имена популярных и широко используемых легитимных библиотек, с целью того, чтобы разработчики случайно установили вредоносную версию из-за опечатки при написании команды установки . [4]

Например, вместо правильного имени пакета requests злоумышленник может зарегистрировать requessts или request. Из-за близкого написания и, возможно, схожего описания, разработчик может не заметить ошибку и установить вредоносный пакет, который будет выполнять свои злонамеренные действия после инсталляции. Исследования показывают, что эта проблема продолжает развиваться, и существуют специализированные системы, такие как TypoSmart, направленные на снижение количества ложных срабатываний при ее обнаружении . Некоторые платформы, как PyPI, начали внедрять собственные механизмы автоматического обнаружения таких попыток во время создания нового проекта . [2]

Обе эти угрозы имеют общую черту: они направлены на манипуляцию с *именами* пакетов, а не с их содержимым. Именно поэтому «SlopShield» фокусируется на анализе происхождения и идентичности зависимостей, а не на сканировании их кода на наличие уязвимостей. Утилита рассматривает ситуацию, когда пакет не существует (slopsquatting) или имеет очень свежий возраст (потенциально создан для атаки), как критическую угрозу, требующую немедленного вмешательства. Аналогично, если имя пакета сильно отличается от имени популярной библиотеки (typosquatting), но при этом используется для ее замены, это также считается тревогой. Решение, предлагаемое «SlopShield», заключается в том, чтобы перехватить эти ситуации на самой ранней стадии — до сборки проекта — и не позволить сборочной системе работать с подозрительными зависимостями.

# Архитектура решения

Архитектура утилиты «SlopShield» была разработана с учетом требований к эффективности, безопасности и простоте интеграции в современные DevOps практики. Центральной идеей архитектуры является позиционирование «SlopShield» в качестве надежного «Gatekeeper» (проверяющего барьера) в конвейере CI/CD (Continuous Integration/Continuous Delivery) . Это означает, [36]
что утилита интегрируется непосредственно в процесс сборки проекта и принимает решение о продолжении или прекращении сборки на основе результатов своего аудита зависимостей. Такой подход является проактивным, поскольку он предотвращает распространение потенциально опасных компонентов на более поздние этапы жизненного цикла разработки, такие как тестирование, развертывание и эксплуатация.
Логика работы «SlopShield» организована в виде многоступенчатого конвейера анализа, который последовательно применяет различные методы для каждой зависимости, указанной в проекте. Конвейер состоит из следующих ключевых этапов:

1. **Парсинг файлов зависимостей:** На первом шаге утилита сканирует и разбирает файлы, содержащие список зависимостей проекта. В настоящее время поддерживаются стандартные форматы для наиболее популярных экосистем: requirements.txt для Python и package.json для Node.js [55] . Этот этап преобразует текстовое описание зависимостей в [56] структурированный список объектов, каждый из которых содержит имя пакета и, при наличии, версионный диапазон.

2. **Проверка по локальному белому списку:** После парсинга каждая зависимость извлекается и проверяется на наличие в локально хранящемся белом списке. Этот список содержит имена пакетов, которые разработчики или администраторы системы считают абсолютно доверенными. Хранение этого списка в зашифрованном виде является важным элементом безопасности, который защищает его от несанкционированного изменения внутри системы . Если пакет [5] находится в белом списке, он считается безопасным, и проверка по нему прекращается.

3. **Детекция фантомов (Anti-Slop):** Это сердцевина инновационности «SlopShield». Для каждой пакета, не найденного в белом списке, утилита выполняет запрос к API соответствующего репозитория (PyPI или npm) для проверки его существования. Здесь реализованы два под-правила:

- **Ответ 404 Not Found:** Если API возвращает статус "Не найдено", это интерпретируется как критическая тревога. Это однозначный признак того, что пакет либо является результатом галлюцинации ИИ (slopsquatting), либо является приватным пакетом, доступ к которому не

настроен в конфигурации CI/CD. В любом из этих случаев сборка блокируется, так как использовать несуществующий пакет невозможно, а использование приватного без прав — небезопасно.

• **Анализ метаданных (проверка по возрасту):** Если пакет существует (API возвращает статус 200 OK), «SlopShield» извлекает его метаданные, в частности, дату загрузки (`upload_time`). Утилита сравнивает эту дату с текущим временем. Если пакет был загружен менее семи дней назад, он помечается как предупреждение (карантин). Этот механизм помогает выявлять свежие, подозрительные пакеты, которые могли быть созданы злоумышленниками непосредственно для атаки на данный проект. Хотя новый пакет не обязательно вредоносен (например, это может быть легитимный выпуск библиотеки), его свежесть делает его высокорискованным.

1. **Детекция typosquatting (Anti-Typo):** На этом этапе для каждой подозрительной зависимости выполняется нечеткий поиск ее имени. Утилита берет имя пакета и сравнивает его с именами из большой базы данных популярных пакетов (например, «Топ-5000»). Для расчета степени схожести используется алгоритм расстояния Дамерау-Левенштейна, который измеряет минимальное количество односимвольных операций (вставки, удаления, замены и транспозиции двух соседних символов), необходимых для преобразования одного слова в другое . Правило [46] детекции выглядит следующим образом: если расстояние между именем искомого пакета и именем популярного пакета меньше или равно 2, и при этом разница в их популярности (например, по количеству скачиваний) составляет более 1000 раз, то искомый пакет помечается как подозрительный typosquat. Это позволяет эффективно находить опечатки, даже если они не являются простыми опечатками одного символа.

2. **Генерация SBOM:** По завершении анализа всех зависимостей, «SlopShield» формирует итоговый отчет в виде SBOM (Software Bill of Materials). Этот отчет содержит всю собранную информацию о каждой зависимости: ее имя, версию, статус (безопасно, подозрительно, вредоносно) и причину такого статуса.

Эта многоступенчатая архитектура обеспечивает комплексный подход к анализу, сочетая внешнюю верификацию через API, анализ временных меток и нечеткий поиск. Такой подход минимизирует количество ложноположительных срабатываний и позволяет утилите принимать обоснованные решения, блокируя сборку только в случаях, когда уровень риска является значительным.

# Технологический стек

Выбор технологического стека для утилиты «SlopShield» был продиктован необходимостью обеспечить высокую производительность, максимальную безопасность и удобство интеграции в существующие рабочие процессы разработки. Каждый компонент выбранной технологии был взвешенно обоснован и

вносит свой вклад в общую надежность и эффективность решения.

| Компонент | Технология | Обоснование |
|---|---|---|
| **Язык программирования** | Python 3.10+ | Выбор Python является стандартом де-факто для разработки инструментов безопасности и CLI-утилит. Его богатая экосистема, простота синтаксиса и мощные библиотеки для работы с сетью, файловой системой и криптографией значительно ускоряют разработку и обеспечивают необходимую гибкость. Версия 3.10+ гарантирует использование современных языковых конструкций и улучшенной производительности. |
| **Сериализация данных** | Google Protocol Buffers (proto3) | Вместо традиционного JSON, «SlopShield» использует бинарный формат Protobuf для генерации SBOM. Это решение было принято по нескольким причинам: 1) **Компактность**: Бинарные данные занимают значительно меньше места, что снижает объем хранимой информации и ускоряет передачу. 2) **Производительность**: Парсинг бинарных форматов обычно происходит быстрее, что критично для CI/CD-пайплайнов, где скорость выполнения шагов важна. 3) **Защита от подделки (Tamper-resistance)**: Бинарный формат гораздо сложнее для ручного редактирования злоумышленником по сравнению с текстовым JSON, что повышает целостность SBOM. 4) **Структурированность**: Строгая схема .proto обеспечивает четкую и стандартизированную структуру данных, что способствует совместимости и дальнейшей обработке . [22] |
| **Криптография** | AES-GCM (библиотека cryptography) | Для защиты генерируемого SBOM используется симметричный алгоритм шифрования AES в режиме Galois/Counter Mode (GCM). Этот режим является современным стандартом, который обеспечивает не только конфиденциальность данных ( confidentiality), но и их целостность ( integrity), что называется [5] [10] аутентифицированным шифрованием с дополнительными данными (AEAD) . Это гарантирует, что SBOM не только не может быть прочитан неавторизованным лицом, но и не может быть изменен без обнаружения. Использование проверенной библиотеки cryptography обеспечивает реализацию криптографических примитивов на безопасном и оптимизированном уровне. |
| **Нечеткий поиск** | Расстояние Дамерау Левенштейна (библиотека Levenshtein) | Для обнаружения typosquatting используется алгоритм расстояния Дамерау Левенштейна. Он эффективно измеряет схожесть строк, учитывая не только замены, но и транспозиции символов, что особенно важно для обнаружения опечаток. Выбор C-оптимизированной библиотеки Levenshtein обеспечивает высокую производительность при работе со списками популярных пакетов, что позволяет проводить сравнения быстро даже при большом количестве [46] [52] зависимостей . |

Таким образом, выбранный технологический стек является не просто набором популярных

инструментов, а продуманной архитектурой, где каждый компонент решает конкретную задачу и вносит свой вклад в общую безопасность и производительность системы. Отказ от JSON в пользу Protobuf и использование AEAD-шифрования в виде AES-GCM являются ключевыми решениями, которые отличают «SlopShield» от многих других инструментов и делают его SBOM более надежным и защищенным.

# Результаты и форматы данных

Основным результатом работы утилиты «SlopShield» является сгенерированный в процессе аудита SBOM (состав программного обеспечения) в защищенном и компактном формате. Этот SBOM представляет собой точный и неискажаемый отчет о составе проекта на момент сборки, а также о результатах анализа каждой зависимости на предмет наличия угроз slopsquatting и typosquatting.

Вместо традиционного, текстового формата (например, JSON), «SlopShield» использует бинарный формат Google Protocol Buffers (Protobuf) версии 3 . [22]
Такой выбор был сделан для достижения трех ключевых целей: повышения производительности, экономии места и усиления защиты от подделки. Бинарный формат значительно компактнее и быстрее парсится, чем его текстовый аналог, что делает SBOM более эффективным для хранения и дальнейшей обработки в автоматизированных системах .
[23]

Структура данных в SBOM определена в .proto файле и состоит из двух основных сообщений: Component и SecureSBOM.

**Схема данных Component:** Это основное сообщение, описывающее отдельную компоненту (зависимость) проекта.

- `name (string, поле 1)`: Имя пакета в формате Package URL (purl), например, `requests`.
- `version (string, поле 2)`: Версия пакета, установленная в проекте, например, `2.31.0`.
- `purl (string, поле 3)`: Полное идентификаторное имя пакета (Package URL), уникально идентифицирующее его в рамках экосистемы.
- `integrity_hash (string, поле 4)`: Хеш (например, SHA256) артефакта пакета, который гарантирует его целостность.

- `status (RiskLevel, поле 5)`: Риск-статус компоненты. Это энумерация, принимающая одно из трех значений: `SAFE` (безопасно), `SUSPICIOUS` (подозрительно, например, пакет новый), `MALICIOUS` (вредоносно, например, пакет не существует или является подтвержденным typosquat). •
  `risk_reason (string, поле 6)`: Текстовое описание, объясняющее, почему компонента была помечена данным статусом. Например, "Package age is less than 7 days" (Пакет возрастом менее 7 дней) или "Detected as typo squatting for popular package 'numpy'" (Обнаружен как typosquatting для популярного пакета 'numpy').

**Схема данных SecureSBOM:** Это контейнерное сообщение, которое оборачивает массив компонентов и содержит метаданные самого отчета.

- `timestamp (int64, field 1)`: Временная метка Unix в формате int64, указывающая на момент времени, когда был проведен аудит. Это позволяет отслеживать актуальность SBOM.
- `author_signature (string, field 2)`: Подпись, которую может добавить оператор системы или CI-система. Это служит дополнительным уровнем атрибуции и подтверждения ответственности за сборку.
- `tool_version (string, field 3)`: Версия утилиты «SlopShield», которая сгенерировала отчет. Это важно для отладки и отслеживания изменений в логике анализа.
- `components (repeated Component, field 4)`: Массив повторяющихся сообщений `Component`, содержащий полный перечень всех зависимостей, проанализированных утилитой.

После генерации бинарный файл SBOM не остается в открытом виде. Он подлежит шифрованию с использованием алгоритма AES-GCM с применением секретного ключа. Этот процесс обеспечивает два уровня защиты: 1. **Конфиденциальность:** Шифрование гарантирует, что содержимое SBOM, содержащее информацию о зависимостях, не может быть прочитан неавторизованным лицом. 2. **Целостность и Аутентичность:** Режим GCM (Galois/ Counter Mode) является режимом аутентифицированного шифрования (AEAD). Это означает, что любой несанкционированный модификация зашифрованного файла SBOM будет обнаружена при его расшифровке. Это полностью защищает от атаки «подмена состава ПО» (SBOM Tampering), которая может произойти, если злоумышленник получит доступ к системе после успешной компрометации и попытается

скрыть следы своих действий, изменив отчет SBOM .
5 10

Таким образом, «SlopShield» предоставляет не просто список зависимостей, а защищенный цифровой сертификат происхождения и безопасности, который можно использовать для аудита, соответствия нормативным требованиям и дальнейшего анализа безопасности, будучи уверенным в его подлинности и полноте.

# Приложения

**Приложение 1: Пример использования утилиты в CI/CD-пайплайне**

```
## .gitlab-ci.yml или .github/workflows/main.yml пример stages:
 - audit
 - build

audit-dependencies:
 stage: audit
 image: python:3.10-slim
 script:
 # 1. Установка SlopShield (предполагается, что он доступен через pip)  - pip install slopshield

 # 2. Запуск аудита зависимостей
   # Флаг --fail-on-error блокирует сборку при обнаружении MALICIOUS пак # Флаг --quarantine-newer-than 7 описывает параметр для карантина нов - slopshield audit --file requirements.txt --fail-on-error --quaranti

 artifacts:
 reports:
 # GitLab CI может использовать этот файл для анализа  sast: slopshield-report.json

build-app:
 stage: build
 needs: ["audit-dependencies"]
 # ... остальные шаги сборки
```

**Приложение 2: Полный протофайл slopushield.proto**

```
syntax = "proto3";
```

```
package slopshield.v1;

option go_package = "github.com/example/slopushield/slopushieldv1;slopushoption java_package = "com.example.slopushield.v1";
option java_outer_classname = "SlopShieldV1";
// Компонента программного обеспечения (пакет)
message Component {
 // Имя пакета, например "requests"
 string name = 1;
 // Версия пакета, например "2.31.0"
 string version = 2;
 // Полный Package URL (purl)
 string purl = 3;
 // Хеш целостности артефакта (SHA256)
 string integrity_hash = 4;

 // Уровень риска компоненты
 enum RiskLevel {
 // Безопасно: Пакет доверен или успешно прошел все проверки.  SAFE = 0;
 // Подозрительно: Пакет новый (возраст < 7 дней) или является опечатк SUSPICIOUS = 1;
 // Вредоносно: Пакет не существует (фантом) или подтвержден как typos MALICIOUS = 2;
 }
 // Текущий статус риска
 RiskLevel status = 5;
 // Текстовое описание причины риска
 string risk_reason = 6;
}

// Защищенный SBOM (Bill of Materials)
message SecureSBOM {
 // Метаданные отчета
 // Временная метка в формате Unix Timestamp
 int64 timestamp = 1;
 // Подпись ответственного лица или системы CI/CD
 string author_signature = 2;
 // Версия утилиты, сгенерировавшей отчет
 string tool_version = 3;
```

```
// Список проанализированных компонент
repeated Component components = 4;
}
```

**Приложение 3: Таблица сравнения подходов к детекции угроз**

| Угроза | Традиционный SCA (например, Snyk) | «SlopShield» |
|---|---|---|
| **Основной принцип** | Поиск известных уязвимостей (CVE) в *существующих* пакетах. | Выявление аномалий происхождения и идентичности пакетов. |
| **Обнаружение Slopsquatting** | Не обнаруживается. Пакет технически «чист» и не имеет CVE. | Обнаруживается. Проверка существования через API (статус 404) является критической угрозой. |
| **Обнаружение Typosquatting** | Может обнаруживать, если typosquat-пакет скопировал уязвимый код из легитимной библиотеки. | Обнаруживается на основе имен. Нечеткий поиск по расстоянию Дамерау-Левенштейна. |
| **Результат** | Список пакетов с известными уязвимостями. | SBOM с оценкой риска (безопасно, подозрительно, вредоносно) для каждой зависимости. |
| **Защита от подмены SBOM** | Обычно JSON, легко подделываем. | Бинарный Protobuf, зашифрованный AES-GCM, защищенный от подделки. |

1.

Container Attack and Defend | DevSecOps Guides https://www.linkedin.com/posts/devsecopsguides_container-attack-and-defend-activity-7351913087345606657-S6js

2. PyPI in 2025: A Year in Review - The Python Package ... https://blog.pypi.org/posts/2025-12-31-pypi-2025-in-review/

3. Compare Packages Between Distributions https://distrowatch.com/dwres.php?resource=compare packages&firstlist=gentoo&secondlist=devuan&firstversions=0&secondversions=0&showall=yes

4. 平台和工具 - 架构师研究会 https://architect.pub/book/export/html/36

5. Release Notes - 《Kong Gateway v3.4 Documentation》 https://www.bookstack.cn/read/kong-3.4-en/ad6b829c24f5ff24.md

6. Beyond CI/CD: A Complete Guide to Application, ... https://dev.to/srinivasamcjf/beyond-cicd-a-complete-guide-to-application-infrastructure-data-and-security pipelines-3112

7. A Time Series Analysis of Malware Uploads to ... https://arxiv.org/pdf/2504.15695 8.

Microsoft cloud security benchmark v2 - DevOps Security https://learn.microsoft.com/en-us/security/benchmark/azure/mcsb-v2-devop-security

9. ENISA THREAT LANDSCAPE 2023 https://www.enisa.europa.eu/sites/default/files/publications/ENISA%20Threat%20Landscape%202023.pdf

10. (PDF) Understanding Data Breach from a Global Perspective https://www.researchgate.net/publication/377865371_Understanding_Data_Breach_from_a_Global_Perspective_Incident_Visualization_and_Data_Protection_Law_Review

11. ESF Labs Limited's Post https://www.linkedin.com/posts/esf-labs limited_cybersecurity-dell-privilegeescalation-activity-7283415810113445888-Xeq1 12.

Slop Squatting: The Hidden AI-Driven Threat Putting ... https://www.linkedin.com/pulse/slop-squatting-hidden-ai-driven-threat-putting-technology-fred-gordy-pejge

13. AI agents' vulnerabilities: A study on insecure MCP client ... https://www.linkedin.com/posts/shuyangwang_from-well-known-to-well-pwned-common vulnerabilities-activity-7379977144657092608-rW7x

14. Measuring LLM Package Hallucination Vulnerabilities https://arxiv.org/html/2501.19012v1
15. Risks of Using Malicious Software Packages https://www.linkedin.com/top-content/technology/cybersecurity-exploit-techniques/risks-of-using-malicious-software packages/
16. C++ vs Python | Joas A Santos https://www.linkedin.com/posts/joas-antonio-dos santos_redteam-programming-shellcoderunner-activity-7288736386461962240-YrNw
17. Topics https://pub.dev/topics
18. Architecting a True Open Source Software Supply Chain ... https://www.linkedin.com/pulse/sovereign-fortress-architecting-true-open-source-software ignisca-ijgrf
19. Simple Index https://mirrors.sustech.edu.cn/pypi/simple/
20. NixOS https://distrowatch.com/table-mobile.php?distribution=nixos&pkglist=true&version=unstable
21. (PDF) A State-of-the-Art Review on Phishing Website ... https://www.researchgate.net/publication/386849281_A_State-of-the art_Review_on_Phishing_Website_Detection_Techniques
22. SBOM Generation Tools in the Python Ecosystem https://arxiv.org/html/2409.01214v1
23. Technology Radar https://www.thoughtworks.com/content/dam/thoughtworks/documents/radar/2022/03/tr_technology_radar_vol_26_en.pdf
24. Vibe Coding with "Claude Code CLI" and "sst/OpenCode" https://www.udemy.com/course/ai-driven-development-with-claude-code-cli-and-opencode/?srsltid=AfmBOopkLU7uFP6GzLR4WK16I4YSAVEUFbMwIwLdzz_vG13SgDEAppzX
25. Is this AES GCM file encryption good practice? https://stackoverflow.com/questions/64867493/is-this-aes-gcm-file-encryption-good-practice
26. Supported algorithm suites in the AWS Encryption SDK https://docs.aws.amazon.com/encryption-sdk/latest/developer-guide/supported

algorithms.html

27. Compromise: A Systematic ... https://
(PDF) AI-Induced Supply-Chain
www.researchgate.net/publication/397443718_AI-Induced_Supply Chain_Compromise_A_Systematic_Review_of_Package_Hallucinations_and_Slopsquat ting_Attacks

28. TypoSmart: A Low False-Positive System for Detecting ... https://arxiv.org/html/ 2502.20528v1

29. Malicious Package Detection in NPM and PyPI using a ... https://dl.acm.org/doi/ 10.1145/3705304

30. Detecting Active and Stealthy Typosquatting Threats in ... https:// www.researchgate.net/publication/ 389510572_Detecting_Active_and_Stealthy_Typosquatting_Threats_in_Package_Regis tries

31. Towards a Statement-Level Taxonomy and Dataset for ... https://www.arxiv.org/pdf/ 2512.12559

32. Typosquatting and Combosquatting Attacks on the Python ... https:// www.researchgate.net/publication/ 344904890_Typosquatting_and_Combosquatting_Attacks_on_the_Python_Ecosystem 33. arXiv:2403.12196v4 [cs.CR] 6 Jan 2025 https://arxiv.org/pdf/2403.12196 34. 人工智能语言大模型性能测试用例设计原创 https://blog.csdn.net/Javachichi/article/ details/154012722

35. mcp-contextforge-gateway https://pypi.org/project/mcp-contextforge-gateway/ 36. internal-developer-platform.pdf https://docs.aws.amazon.com/pdfs/prescriptive guidance/latest/internal-developer-platform/internal-developer-platform.pdf

37. Implement column-level encryption to protect sensitive data ... https:// aws.amazon.com/blogs/big-data/implement-column-level-encryption-to-protect sensitive-data-in-amazon-redshift-with-aws-glue-and-aws-lambda-user-defined functions/

38. TinyCrypt Cryptographic Library - Technical Documentation https:// docs.nordicsemi.com/bundle/ncs-1.7.1/page/zephyr/guides/crypto/tinycrypt.html

39.

How to encrypt String in Java https://stackoverflow.com/questions/1205135/how-to-encrypt-string-in-java

40. Different typosquatting attacks to know of - for a secure ... https://dev.to/sumstrm/different-typosquatting-attacks-to-know-of-for-a-secure-supply-chain-4j70

41. Python: How to Encrypt and Decrypt with AES https://dev.to/ankitmalikg/python-how-to-encrypt-and-decrypt-with-aes-4h1k

42. Arxiv今日论文| 2025-12-30 - 闲记算法 http://lonepatient.top/2025/12/30/arxiv_papers_2025-12-30

43. 2025 CISSP Domain Objectives | PDF | Computer Security https://www.scribd.com/document/903382829/2025-CISSP-Domain-Objectives

44. ShmooCon dataset (v02) - Greg Conti https://www.gregconti.com/publications/202507-ShmooCon-data-v02.xlsx

45. Encrypt and decrypt a string in C#? [closed] https://stackoverflow.com/questions/202011/encrypt-and-decrypt-a-string-in-c

46. A Comparative Analysis of Python Text Matching Libraries https://www.researchgate.net/publication/390846511_A_Comparative_Analysis_of_Python_Text_Matching_Libraries_A_Multilingual_Evaluation_of_Capabilities_Performance_and_Resource_Utilization

47. python - Better fuzzy matching performance? https://stackoverflow.com/questions/21408760/better-fuzzy-matching-performance

48. How To Detect Typosquatting With Python https://www.activestate.com/blog/how-to-detect-typosquatting-with-python/

49. RapidFuzz https://pypi.org/project/RapidFuzz/

50. RapidFuzz · PyPI https://pypi.org/project/RapidFuzz/2.0.14/

51. RapidFuzz · PyPI https://pypi.org/project/RapidFuzz/0.6.3/

52. RapidFuzz · PyPI https://pypi.org/project/RapidFuzz/0.1.0/

53. Owasp MSTG-1.2 | PDF | Web Application https://www.scribd.com/document/528568219/OWASP-MSTG-1-2

54. Integrating OWASP Dependency-Track into GitLab CI/CD ... https://stackoverflow.com/questions/72911670/integrating-owasp-dependency-track-into gitlab-ci-cd-pipeline

55. CycloneDX SBOM Generation Tool for Python - Read the Docs https://cyclonedx-bom tool.readthedocs.io/

56. CycloneDX Python SBOM生成工具常见问题解决方案 https://blog.csdn.net/gitblog_00207/article/details/144578251